Practical 7:

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

```
Student@DESKTOP-JJ8CFNB MINGW64 ~
$ nano banker.c

Student@DESKTOP-JJ8CFNB MINGW64 ~
$ gcc banker.c -o banker
```

Code:

```
~
  GNU nano 8.7
#include <stdio.h>

#define P 5   // Number of processes
#define R 4   // Number of resources

int main() {
    int alloc[P][R], max[P][R], need[P][R];
    int available[R];
    int finish[P] = {0};
    int safeSeq[P];
    int i, j, k;
    int count = 0;

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Max Matrix:\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Available Resources:\n");
    for(i = 0; i < R; i++) {
        scanf("%d", &available[i]);
    }

    // Calculate Need Matrix
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    printf("\nNeed Matrix:\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
```

```
    // Banker's Algorithm
    while(count < P) {
        int found = 0;
        for(i = 0; i < P; i++) {
            if(finish[i] == 0) {

                for(j = 0; j < R; j++) {
                    if(need[i][j] > available[j])
                        break;
                }

                if(j == R) {
                    for(k = 0; k < R; k++) {
                        available[k] += alloc[i][k];
                    }

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("\nSystem is NOT in safe state (Deadlock).\n");
            return 0;
        }
    }

    printf("\nSystem is in SAFE state.\nSafe sequence is:\n");
    for(i = 0; i < P; i++) {
        printf("P%d ", safeSeq[i]);
    }

    return 0;
}
```

Output:

```
System is in SAFE state.
Safe sequence is:
P1 P2 P3 P4 P0
Student@DESKTOP-JJ8CFNB MINGW64 ~
$
```